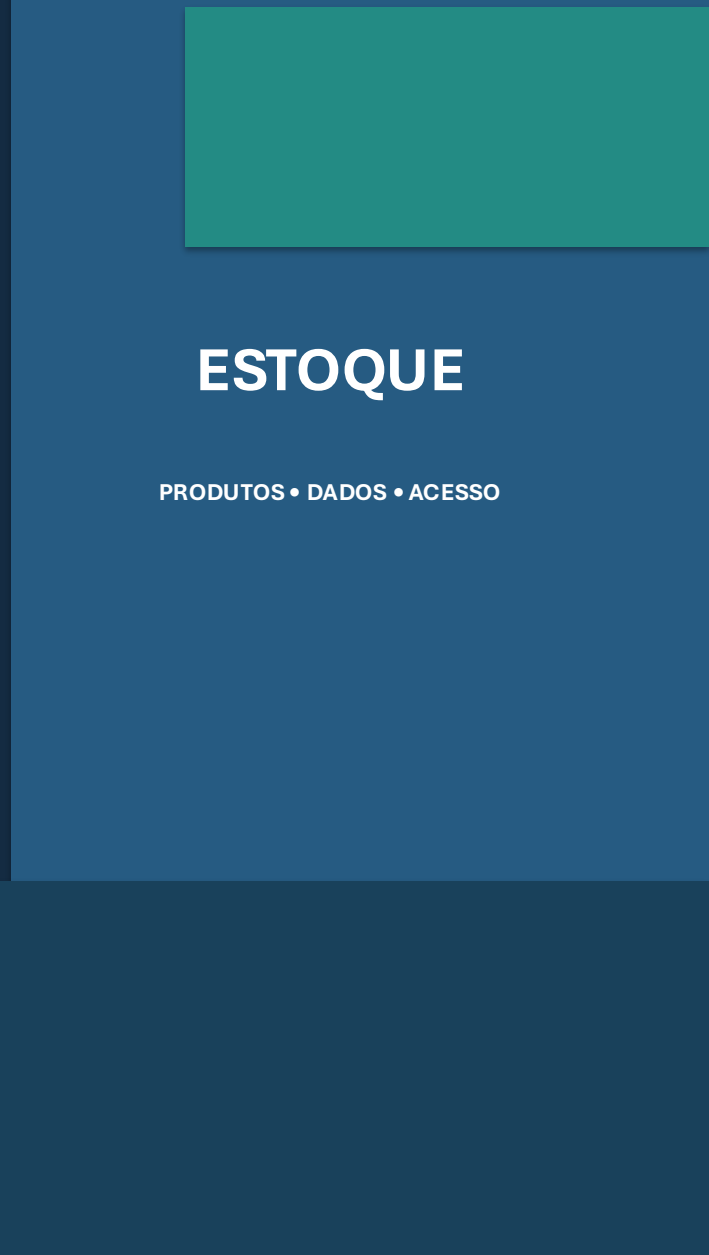
TRABALHO PRÁTICO • DESENVOLVIMENTO WEB

Sistema Web de Controle de Estoque

Aplicação desenvolvida com Spring Boot

Gabriel Gaete dos Santos

Engenharia de Software | FACOM – UFMS



1. Problema e proposta

Por que desenvolver um sistema de controle de estoque?

CENÁRIO DO PROBLEMA

- Registros manuais podem gerar desorganização.
- É difícil consultar rapidamente preços e quantidades.
- Alterações no estoque precisam ser controladas.

PROPOSTA DO PROJETO

- Centralizar os produtos em uma aplicação web.
- Facilitar a consulta das informações do estoque.
- Restringir alterações conforme o perfil de acesso.

2. Objetivo e escopo do sistema

O que a aplicação se propõe a resolver

CONSULTAR

Visualizar a lista de produtos cadastrados.

CADASTRAR

Adicionar produtos com nome, categoria, preço e quantidade.

ATUALIZAR

Editar informações de um produto existente.

EXCLUIR

Remover um produto do cadastro.

Entidade principal: Produto • Operações realizadas por meio de páginas web.

3. Atores e casos de uso

Papéis que utilizam o sistema e operações disponíveis

ATORES

V — **VISITANTE**
Acessa a tela de login

U — **USUÁRIO COMUM**
Consulta os produtos

A — **ADMINISTRADOR**
Consulta e gerencia produtos

CASOS DE USO

UC01
Realizar login
Visitante

UC02
Visualizar produtos
Usuário + Admin

UC06
Realizar logout
Usuário + Admin

UC03
Cadastrar produto
Somente Admin

UC04
Editar produto
Somente Admin

UC05
Excluir produto
Somente Admin

As operações de cadastro, edição e exclusão são restritas ao perfil ADMIN.

4. Requisitos funcionais

O que o sistema deve fazer para atender às necessidades do controle de estoque

RF01 A RF05 • ACESSO E OPERAÇÕES

RF01	Permitir login com nome de usuário e senha.
RF02	Exibir os produtos para usuários autenticados.
RF03	Permitir ao administrador cadastrar produtos.
RF04	Permitir ao administrador editar produtos existentes.
RF05	Permitir ao administrador excluir produtos.

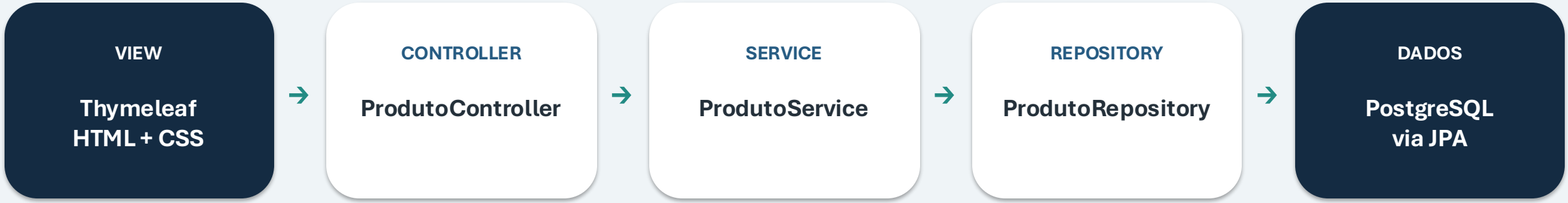
RF06 A RF10 • CONTROLE E DADOS

RF06	Permitir que usuários autenticados encerrem a sessão.
RF07	Restringir cadastro, edição e exclusão ao perfil ADMIN.
RF08	Exigir nome e categoria no cadastro e na edição.
RF09	Impedir preço e quantidade com valores negativos.
RF10	Armazenar os dados dos produtos no PostgreSQL.

As validações de nome, categoria, preço e quantidade são aplicadas no cadastro e na edição.

5. Arquitetura adotada

Opção A — Spring MVC com renderização no servidor



O navegador solicita uma página; o Controller coordena a operação, o Service aplica a lógica e o Repository acessa o banco.

6. Tecnologias utilizadas

Ferramentas identificadas no projeto

Java 17	Linguagem da aplicação
Spring Boot 3.5.5	Estrutura e inicialização
Spring MVC + Thymeleaf	Rotas e páginas HTML
Spring Data JPA / Hibernate	Persistência e mapeamento
PostgreSQL	Banco de dados relacional
Spring Security	Autenticação e autorização
Maven	Gerenciamento de dependências

Projeto configurado para executar na porta 8081.

7. Modelo de dados e persistência

A entidade Produto representa os registros do estoque

PRODUTO

id : Long (chave primária)

nome : String

categoria : String

preco : BigDecimal

quantidade : Integer

COMO OS DADOS SÃO SALVOS

- Produto é uma entidade JPA (@Entity).
- ProdutoRepository estende JpaRepository.
- Hibernate realiza o mapeamento para tabelas do PostgreSQL.
- A configuração JPA utiliza ddl-auto=update.

Banco: controle_estoque

8. Organização do código

Separação de responsabilidades em camadas

model / Produto

Define os atributos e as validações dos dados.

↓

repository / ProdutoRepository

Oferece operações de acesso ao banco com JpaRepository.

↓

service / ProdutoService

Centraliza operações de listar, buscar, salvar e excluir.

↓

controller / ProdutoController

Recebe requisições e direciona para as páginas.

↓

config / SegurancaConfig

Configura autenticação, autorização e logout.

9. CRUD de produtos

Fluxo principal de gerenciamento do estoque

CREATE	READ	UPDATE	DELETE
Cadastrar	**Listar e consultar**	**Editar**	**Excluir**
/produtos/novo /produtos/salvar	/produtos	/produtos/editar/{id} /produtos/atualizar	/produtos/excluir/{id}

Após salvar, atualizar ou excluir, o sistema redireciona para a listagem de produtos.

10. Controle de acesso — Spring Security

Dois papéis definem o que cada perfil pode fazer

ROLE_USER | Usuário comum

- Acessa a listagem de produtos.
- Visualiza as informações cadastradas.
- Não pode executar operações de alteração.

ROLE_ADMIN | Administrador

- Visualiza os produtos.
- Pode cadastrar, editar e excluir.
- Rotas administrativas protegidas por @PreAuthorize.

11. Acessibilidade e interface

Práticas observadas nas páginas HTML e pontos de verificação

IMPLEMENTADO NO CÓDIGO

- Idioma da página definido como pt-BR.
- Uso de header, nav, main e footer em páginas principais.
- Rótulos label associados aos campos do formulário.
- Tabela com caption e cabeçalhos.
- Mensagens de erro com role=alert e status.

VALIDAR NA DEMONSTRAÇÃO

- Contraste de texto e botões conforme WCAG AA.
- Navegação completa pelo teclado e foco visível.
- Leitura dos formulários com leitor de tela.
- Responsividade em desktop, tablet e celular.

12. Decisões técnicas e participação

Justificativas e contribuição no desenvolvimento

DECISÕES DO PROJETO

- Tema escolhido por representar uma necessidade comum de pequenos comércios.
- Thymeleaf foi adotado para integrar as páginas ao Spring MVC.
- PostgreSQL oferece persistência relacional entre execuções.
- A separação em camadas facilita manutenção e evolução.

13. Demonstração e conclusão

Roteiro sugerido para executar o sistema ao vivo

1. Acessar a tela inicial e realizar login como usuário comum.
2. Mostrar a listagem e comprovar que as ações administrativas não estão disponíveis.
3. Sair e entrar como administrador.
4. Cadastrar um produto, editar seus dados e excluí-lo.
5. Conferir no banco se as alterações foram persistidas.

Conclusão: o projeto reúne interface web, CRUD, persistência relacional e controle de acesso em uma estrutura organizada.

Obrigado!